

PEA: Parallel Evolutionary Algorithm by Separating Convergence and Diversity for Large-Scale Multi-Objective Optimization

Huangke Chen[§], Xiaomin Zhu[§], Witold Pedrycz[†], Shu Yin[‡], Guohua Wu[§] and Hui Yan[§]

[§]College of System Engineering,

National University of Defense Technology, Changsha 410073, P. R. China,

{hkchen, xmzhu, guohuawu, yanhui13}@nudt.edu.cn

[†]Department of Electrical and Computer Engineering,

University of Alberta, Edmonton, AB T6G 2V4, Canada,

wpedrycz@ualberta.ca

[‡]SIST, ShanghaiTech University, Shanghai, China, yinshu@shanghaitech.edu.cn

Abstract—Running evolutionary algorithms in parallel is an intuitive way to speed up the process of solving large-scale multi-objective optimization problems, which have hundreds or thousands of decision variables. However, the framework of the existing multi-objective evolutionary algorithms seriously limits their parallelization. During each iteration, the environmental selection operators present in the existing framework need to collect and compare all the candidate solutions to balance the convergence and diversity, thus dividing the whole evolutionary process into a series of dependent sub-processes and resulting in frequent data transmission. To address this issue, we propose a novel parallel framework that separates the environmental selection operator from the entire evolutionary process, evidently removing the dependencies among sub-processes and reducing the data transmission. On the basis of the parallel framework, a new parallel evolutionary algorithm, namely *PEA*, is designed. In *PEA*, the convergence is achieved by a series of independent sub-populations, and the diversity is merely emphasized at the converged solutions from each sub-population, which is helpful for avoiding that the environmental selection operator limits the parallelization of the algorithm. Moreover, a new environmental selection strategy is proposed to improve the diversity without considering the convergence. To assess the performance of the proposed *PEA*, we compare it with five representative multi-objective evolutionary algorithms in terms of both the convergence and diversity. The performance of the parallel framework is also analyzed by comparing with two existing parallel models. The experimental results demonstrate the superiority of the proposed parallel algorithms in terms of the convergence, diversity, and speedup.

Index Terms—Parallel evolutionary computation, evolutionary algorithm, large-scale, multi-objective.

1. Introduction

Optimization problems encountered in real-world applications, such as wireless sensor networks [1], cloud computing [2], reservoir flood control [3] and the like, often involve

multiple conflicting objectives. Such optimization problems are called multi-objective optimization problems (MOPs), which can be formulated as follows [4], [5], [6].

$$\begin{aligned} \min \quad & F(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})], \\ \text{s.t.} \quad & \mathbf{x} \in \Omega, \end{aligned} \quad (1)$$

where $\Omega \subseteq \mathbb{R}^n$ and $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \Omega$ denote decision space and decision vector, respectively. The parameter n represents the dimensionality of the decision space, i.e., the count of decision variables. The objective function is formulated as F , i.e., $F: \mathbb{R}^n \rightarrow \mathbb{R}^m$, and m represents the number of objectives. Besides, the function $f_i(\mathbf{x})$, $\forall i \in \{1, 2, \dots, m\}$ stands for a mapping from Ω to $\mathbb{R}$, i.e., $f_i: \Omega \rightarrow \mathbb{R}$. Generally, when the number of decision variables is larger than 100, the MOPs are regarded as large-scale MOPs [6], [7].

Due to the conflicting nature of the objectives present in MOPs, improving one objective typically cannot be achieved without the deterioration of at least one of the remaining objectives [8], [9]. Thus, there rarely exists a single solution being capable of making all the objectives being optimal [10], and a number of Pareto-optimal solutions are founded for each MOP. For two feasible solutions $\mathbf{x}_1, \mathbf{x}_2 \in \Omega$ considered in an MOP, we say that $\mathbf{x}_1$ dominates $\mathbf{x}_2$ (denoted as $\mathbf{x}_1 \prec \mathbf{x}_2$), if all the objectives of $\mathbf{x}_1$ are not larger than those of $\mathbf{x}_2$ and at least one objective of $\mathbf{x}_1$ is less than that of $\mathbf{x}_2$. A solution $\mathbf{x}^* \in \Omega$ is Pareto optimal if there exists no solution in the decision space that dominates it. In addition, all the Pareto-optimal solutions constitute the Pareto set (*PS*) in the decision space, and the Pareto-front (*PF*) in the objective space.

Evolutionary algorithms (EAs) are widely perceived to be capable of producing a set of non-dominated solutions that are as uniformly distributed over and close to the *PF* as possible, which is helpful for providing insights into the compromises emerging among the objectives for the MOPs. Driven by the practical demands encountered in various applications [11], [12], multi-objective evolutionary algo-

gorithms (MOEAs) have attracted intensive attention and were rapidly developed resulting in a considerable number of interesting studies [8], [10], [13]. MOEAs have demonstrated their effectiveness in solving various MOPs, however, they need hundreds of iterations or more to exploit the search space, and are unable to provide satisfactory results within a reasonable time [14]. This is especially pronounced when the search space explodes exponentially with the increasing number of decision variables, and the search space involves a huge number of local optima [15].

Operating algorithms in parallel is an intuitive way to reduce their execution time [14], [16], [17]. However, the framework of traditional MOEAs, which is summarized in Fig. 1, limits their parallel operations. The MOEA firstly initializes a population (IP), then iterates the following two main procedures until the termination condition is reached: 1) generating new population (GP) using population variation (i.e., crossover and mutation operators) and 2) environmental selection (ES). For the deficiency of the framework in parallelism, the reason is three-fold. Firstly, to select elite solutions for next iteration, the environmental selection operator needs to collect and compare all the new solutions and original solutions, which divides the whole evolutionary process into a series of dependent sub-processes. Secondly, the execution time of each sub-process is very short (about a few milliseconds), and thus frequent data interaction for selecting elite solutions is inevitable, which leads to considerable time overhead of data transmission. Thirdly, what is worse, the environmental selection operators are often time consuming [15]. For four representative algorithms (i.e., NSGA-III, MOEA/D, BiGE and GrEA) solving the problem LSMOP1 with 3-objectives [4], Fig. 2 summarizes their ratios of time of environmental selection operators during each iteration, and the ratio ranges from 8% to 72%.

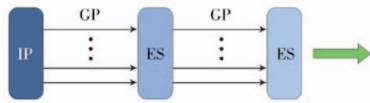


Figure 1. The Framework of Traditional MOEAs.

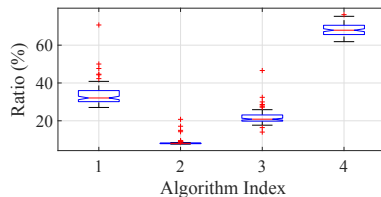


Figure 2. The Time Ratios of Environmental Selection.

To address the aforementioned issues in the traditional MOEAs, this work mainly makes the following technical contributions.

- We propose a novel parallel framework that separates the environmental selection operator from the

whole evolutionary process, which is helpful for removing the dependencies among sub-processes and reducing the data transmission.

- On the basis of the parallel framework, we design a novel parallel evolutionary algorithm, namely *PEA*, to solve multi-objective optimization problems with large-scale variables. Unlike traditional MOEAs that balance convergence and diversity during each iteration, the proposed *PEA* separates them. The convergence is performed by a series of independent sub-populations that do not need to exchange data among themselves. Thus, the time overhead of exchanging data is substantially reduced. Moreover, the environmental selection operator for improving diversity is merely emphasized on converged solutions from each sub-population, rather than all the solutions, which further decreases the time of selecting solutions.
- A novel environmental selection strategy is proposed to improve the population diversity by considering the converged solutions.
- Extensive experiments are conducted to compare the performance of the proposed *PEA* with five representative algorithms in terms of both the convergence and diversity. Also, the speedup of the proposed parallel framework is compared with two widely-used parallel models, i.e., population-parallel and dimension-parallel models. The results demonstrate the superiorities of *PEA* with respect to the convergence, diversity, and speedup.

The paper is organized as follows. The proposed *PEA* is detailed in Section 2. Section 3 covers extensive comparative study to demonstrate the performance of the proposed *PEA*. Section 4 summarizes the recent works of parallel and large-scale multi-objective evolutionary algorithms. Finally, the conclusions and future work are given in Section 5.

2. Algorithm Design

In this section, we detail the proposed parallel framework, followed by the description of parallel algorithms.

2.1. The Proposed Parallel Framework

During each iteration, the traditional MOEAs can be divided into two steps: population evolution and environmental selection. The second step focuses on balancing convergence and diversity, and is much more time consuming compared with the first step. What is worse, the environmental selection often needs to compare candidate solutions in the combined population, and divide the whole evolutionary process into a series of dependent sub-processes, which severely limits their parallelization. Unlike them, we propose a novel parallel framework to separate environmental selection from the evolutionary process, which removes dependencies among sub-processes and is beneficial to the parallelization of evolutionary algorithms. The proposed parallel framework is illustrated in Fig. 3.

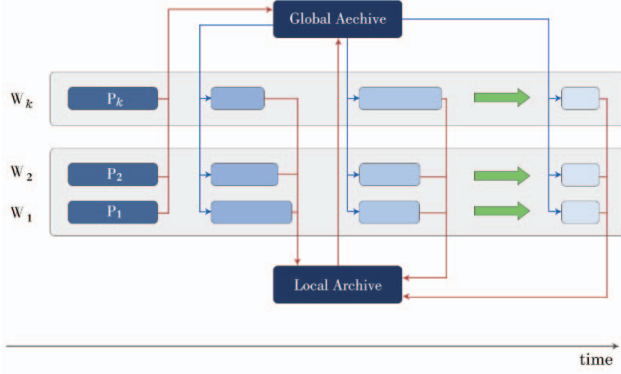


Figure 3. Parallel Framework for Evolutionary Algorithm.

As shown in Fig. 3, the proposed parallel framework contains a series of subpopulations (represented as rectangles), a local archive and a global archive. All the subpopulations are highly parallel, and each one will evolve to convergence on a worker without communicating with any other subpopulations. The symbol $W_h \in \{W_1, W_2, \dots, W_k\}$ denotes the h -th worker, and a worker refers to a core in a CPU. For the first batch of sub-populations, denoted as $p_1, p_2, \dots, p_k$ in Fig. 3, when they converge, a solution with the best fitness in each sub-population will be selected and directly transferred to the global archive. Then, once a worker becomes idle, a new subpopulation will be generated for it based on the obtained solutions in the global archive. In order to avoid updating the global archive too frequently, when a subpopulation (except the first batch) converges, the solution with the best fitness will be transferred to the local archive. When the number of solutions in the local archive reaches the set threshold, all the solutions in the local archive will be transferred to the global archive. At this moment, the environmental selection operator will be performed to all the solutions in the global archive.

Moreover, the proposed framework separates the convergence and diversity, and the evolution of each subpopulation merely pursue the convergence while the environmental selection in global archive merely focuses on the diversity.

To implement the proposed parallel framework, the following three problems should be addressed: how to accelerate each sub-population converge to the PF , how to generate a new subpopulation on the basis of the obtained solutions, and how to improve the diversity of the global archive. The next subsection, we will introduce the relevant strategies to solve these problems.

2.2. Algorithm Description

In the proposed algorithm, the decision variables are classified into two categories: convergence-related and diversity-related variables, which is similar to that in work [6]. The symbols $\mathbf{I}_c$ and $\mathbf{I}_d$ are used to represent the indexes of the convergence-related and diversity-related variables,

respectively. As each subpopulation merely focuses on convergence, they will evolve the convergence-related variables continuously until they converge, and keep diversity-related variables unchanged. The evolutionary process of each subpopulation is described as Algorithm 1.

Algorithm 1: Evolutionary process for subpopulations

Input: The subpopulation P_i ; $\mathbf{I}_c$; group size (s); lower bound of variance (Δ);

Output: Individual q with best fitness in P_i ;

```

1 while true do
2    $G \leftarrow \emptyset$ ;
3    $I(1 : |\mathbf{I}_c|) \leftarrow \text{randperm}(|\mathbf{I}_c|)$ ;
4   for  $j = 1 \rightarrow |\mathbf{I}_c|/s$  do
5      $l \leftarrow (j-1) \times s + 1$ ;  $u \leftarrow \min\{j \times s, |\mathbf{I}_c|\}$ ;
6      $\text{subG} \leftarrow I(l : u)$ ;
7      $G \leftarrow G \cup \{\text{subG}\}$ ;
8   foreach  $\text{subG} \in G$  do
9      $\text{subP} \leftarrow P_i(:, \mathbf{I}_c(\text{subG}))$ ;
10     $\text{subP}' \leftarrow \text{Employ DE and PM on subP}$ ;
11     $nP \leftarrow P_i$ ;
12     $nP(:, \mathbf{I}_c(\text{subG})) \leftarrow \text{subP}'$ ;
13    for  $j = 1 \rightarrow |P_i|$  do
14      if  $\text{Fit}(nP^j) < \text{Fit}(P_i^j)$  then
15         $P_i^j \leftarrow nP^j$ ;
16  if  $\text{var}_{p_i^j \in P_i}(\text{Fit}(p_i^j)) < \Delta$  then
17     $q \leftarrow \text{The individual with best fitness in } P_i$ ;
18    break;
```

As shown in Algorithm 1, in each iteration, all the convergence-related variables are randomly divided into many groups with size s (Lines 2-7). The function $\text{randperm}(|\mathbf{I}_c|)$ is used to generate a vector containing a random permutation of the integers from 1 to $|\mathbf{I}_c|$. For instance, the result of $\text{randperm}(6)$ might be [2 4 5 6 1 3]. After grouping the variables, the variables within different groups will be evolved in sequence (Lines 8-15). For the variables in each group subG , the differential evolution (DE) [18], [19] and polynomial mutation (PM) [20] will be employed to produce new values for these variables (Line 10). Then, the new values of these variables in the group subG will replace the corresponding values of these variables to generate a new subpopulation nP (Line 12). After that, the solutions in subpopulation P_i will be updated if the new solutions in the new subpopulation nP have better fitness (Lines 14-15). The fitness $\text{Fit}(p)$ of a solution p is defined as the sum of all its objective values, and the symbols nP^j and P_i^j denote the j -th solution in the new subpopulation nP and the subpopulation P_i , respectively. At the end of each iteration, the subpopulation P_i will be checked for convergence, and we define that a subpopulation is converged if the variance of the fitnesses of all the solutions within it is less than the lower bound Δ (Line 16). After the subpopulation converges, the solution with

the best fitness will be returned (Line 17).

In the proposed parallel framework, once a subpopulation converges, a new subpopulation will be generated to search new solutions on the basis of the obtained solutions in the global archive. The strategy for generating a new subpopulation is illustrated in Algorithm 2.

Algorithm 2: Generate a new subpopulation

Input: The archive A ; $\mathbf{I}_d, \mathbf{I}_c$; size of subpopulation ($N1$);
Output: A new subpopulation P_i ;

```

1  $P \leftarrow$  Employ DE and PM on  $A$ ;
2  $maxA \leftarrow 0$ ;  $s \leftarrow 0$ ;
3 for  $j = 1 \rightarrow |P|$  do
4    $minA \leftarrow +\infty$ ;
5   for  $k = 1 \rightarrow |A|$  do
6      $Angle(j, k) \leftarrow$  obtain the acute angle
       between  $P_j$  and  $A_k$ ;
7     if  $Angle(j, k) < minA$  then
8        $minA \leftarrow Angle(j, k)$ ;
9   if  $maxA < minA$  then
10     $maxA \leftarrow minA$ ;
11     $s \leftarrow j$ 
12 for  $j = 1 \rightarrow |P|$  do
13    $P(j, \mathbf{I}_d) \leftarrow P(s, \mathbf{I}_d)$ ;
14  $P_i \leftarrow$  Select the first  $N1$  individuals with the best
    fitness from  $P$ ;
```

As described in Algorithm 2, the differential evolution (DE) [18] and polynomial mutation (PM) [20] operators are firstly implemented in the solutions in the global archive A to generate a new population P (Line 1). Note that the size of P is many times (e.g., 10) of the size of a subpopulation, which is helpful to generate better subpopulation in terms of both convergence and diversity. Then, each solutions in P is associated with the minimal acute angle between it and all the solutions in A (Lines 5-8), and the solution with the maximal associated acute angle is chosen (Lines 9-11). As the evolution of a subpopulation merely emphasizes on the convergence, the diversity-related variables of all the solutions in the subpopulation is set as that of the selected solution (Lines 12-13). After that, the first $N1$ solutions with the best fitness are selected as the subpopulation (Line 14).

Since the convergence and diversity are separated in the proposed parallel framework and the convergence is performed by each subpopulation, the environmental selection strategy for global archive focuses on the population diversity. The pseudocode of the proposed environmental selection strategy is described in Algorithm 3.

As shown in Algorithm 3, the proposed environmental selection strategy firstly removes the dominated solutions in the global archive A , and all the non-dominated solutions are placed in the set Q (Line 1). Then, if the number of non-dominated solutions in Q is less than the population size N (Line 2), all the non-dominated solutions are retained (Lines

Algorithm 3: Environmental Selection Strategy

Input: The archive A ; population size (N);
Output: The update archive A ;

```

1  $Q \leftarrow$  Remove the dominated solutions in  $A$ ;
2 if  $|Q| \leq N$  then
3    $A \leftarrow Q$ ;
4   return;
5  $P \leftarrow \{e_1, e_2, \dots, e_m\}$ ;
6  $Q \leftarrow Q \setminus \{e_1, e_2, \dots, e_m\}$ ;
7 while  $|P| < N$  do
8    $maxD \leftarrow 0$ ;  $s \leftarrow 0$ ;
9   for  $j = 1 \rightarrow |Q|$  do
10     $minD \leftarrow +\infty$ ;
11    for  $k = 1 \rightarrow |P|$  do
12       $dv \leftarrow (\sum_{l=1}^m (|f_l(Q_j) - f_l(P_k)|^p))^{\frac{1}{p}}$ ;
13      if  $dv < minD$  then
14         $minD \leftarrow dv$ ;
15    if  $maxD < minD$  then
16       $maxD \leftarrow minD$ ;
17       $s \leftarrow j$ ;
18     $P \leftarrow P \cup \{Q_s\}$ ;
19     $Q \leftarrow Q \setminus \{Q_s\}$ ;
20  $A \leftarrow P$ ;
```

3-4). Otherwise, the following steps will be performed to select N non-dominated solutions from Q . All the extreme solutions in Q , denoted as $e_1, e_2, \dots, e_m$, are selected (Line 5), and removed from the Q (Line 6). Note that a solution has the minimal value of the objective function is regarded as extreme solution. Then, a heuristic strategy is proposed to select solution from set Q one by one. During each iteration, each solution in Q is assigned the minimal dissimilar value between it and all the solutions in P (Lines 11-14), and then the solution with the maximal associated dissimilar value is selected (Lines 15-17). Then, the selected solution is added to set P (Line 18) and removed from the set Q (Line 19). Once the number of solutions in P reaches the population size N , all the solutions in P are used to replace all the solutions in the global archive A (Line 20).

3. Experimental Studies

To evaluate the performance of the proposed parallel evolutionary algorithm *PEA* in solving large-scale multi-objective optimization problems, we compare it with five state-of-the-art many-objective optimization algorithms: *LMEA* [6], *MOEA/DVA* [7], *NSGA-III* [9], *MaOEA-R&D* [21] and *BiGE* [5]. The brief descriptions of the five algorithms are given as follows.

LMEA uses the k -means method to cluster the decision variables into convergence-related and diversity-related variables on the basis of the angles between sample objective vectors and the convergence direction. Then, this algorithm further divides the convergence-related variables into

many subcomponents, and variables in each subcomponent are interactional. During each iteration, *LMEA* evolves the diversity-related variables and the variables in each subcomponent, respectively.

MOEA/DVA is developed on the basis of decision variable analysis. First, it separates the decision variables into convergence-related and diversity-related variables. Then, the convergence-related variables are divided into many low-dimensional subcomponents. After that, the variables in different subcomponents are optimized one by one.

NSGA-III follows the framework of NSGA-II [20], where the crowding distance operator in the NSGA-II is replaced with a reference vector based strategy for diversity maintenance. To be specific, the strategy associates each candidate solution in the last accepted front to a reference vector with the minimal perpendicular distance, and then first selects the candidate solutions with shorter perpendicular distance to the associated reference vectors.

MaOEA-R&D first pushes the whole population quickly converging to several solutions near the Pareto front; then, uses the diversity improvement strategy to generate more solutions based on the obtained solutions to improve the diversity.

BiGE employs two indicators in terms of convergence and diversity to evaluate each candidate solution, and then divides all the solutions into a series of non-dominated fronts based on these two metrics. For the candidate solutions in the last accepted fronts, they are selected randomly.

All the five comparison algorithms above and the proposed algorithm *PEA* are coded in Matlab. The source codes of the five comparison algorithms are available in the platform PlatEMO [22]¹.

3.1. Experimental settings

Benchmark Problems: To compare the performance of the six algorithms, the following ten benchmark MOPs coming from two test suites are utilized: 1) LSMOP1-LSMOP3 [4]; 2) MaF1-MaF7 [23]. The objective number of these problems are set as 3, 5, 8 and 10. For test problems LSMOP1-LSMOP3, the number of their decision variables are relevant to their objective number, and are 307, 514, 817 and 1039 for 3-, 5-, 8- and 10-objectives, respectively. Regarding the problems MaF1-MaF7, the number of decision variables are set as 500. In this paper, an MOP with a specific number of objectives is called a test instance, e.g., problem MaF1 with 3 objectives.

Performance Metric: The inverted generational distance (*IGD*) [24] is utilized to measure the output population with respect to both convergence and diversity. Let P^* be a set of uniformly distributed Pareto optimal points on the *PF* in the objective space. Let P be the output population obtained by an algorithm. The *IGD* from P^* to P is defined as

$$IGD(P^*, P) = \frac{\sum_{v \in P^*} d(v, P)}{|P^*|}, \quad (2)$$

1. <http://bimk.ahu.edu.cn/index.php?s=/Index/Software/index.html>

where $d(v, P)$ is the minimum Euclidean distance between v and the points in P . If P^* is large enough, $IGD(P^*, P)$ could measure both convergence and diversity of P . From the formula (2), we can observe that the output population approximating the *PF* better will have lower *IGD* value, which also means the corresponding algorithm has the better performance. To calculate the metric *IGD*, we use default settings to select about 8,000 uniformly distributed reference Pareto optimal points P^* for these benchmarks.

General setup: To fairly compare the performance of the six algorithms, their population size and termination condition are set the same on each test instance.

1) *Population size:* The population sizes are set on the basis of the objective number of the MOPs being solved, namely, 78 for three-objective instances, 85 for five-objective instances, 128 for eight-objectives instances and 230 for ten-objectives instances, respectively.

2) *Termination condition:* The maximal number of function evaluations is used as the termination condition for all the six algorithms. For each test instances, the maximal number of function evaluations is set on the basis of the number of the decision variables n , and is $10,000 \times n$. All the six algorithms are run for 30 times on each test instance to obtain the statistical characterization (mean and standard deviation) of the *IGD* values.

The detailed settings of the *PEA* are as follows: the number of parallel subpopulations is 8; the size of each subpopulation N_1 is 20; the lower bound of variance (Δ) for the convergence condition of each subpopulation is $1e - 6$; the group size is 100.

3.2. Results and Analysis

For the six algorithms, the mean and standard deviation (in parentheses) of their *IGD* values are reported in Table 1. Similar to [6], [7], this paper also uses the Wilcoxon Rank-Sum test with $\alpha = 0.05$ to test the significant differences between the metric values obtained by *PEA* and that of the others. The symbols $-$, $+$ and $\approx$ are used to represent that the *IGD* value obtained by the comparative algorithm is significantly worse than, better than, and similar to that obtained by the proposed *PEA*, respectively. Besides, the best metric values (i.e., the lowest *IGD*) are highlighted in gray for each test instance.

As illustrated in Table 1, among the 40 test instances, the proposed *PEA* significantly outperforms *LMEA*, *MOEA/DVA*, *NSGA-III*, *MaOEA-R&D* and *BiGE* on 33, 37, 38, 39 and 40, respectively. In summary, with respect to the *IGD* indicator, algorithm *PEA* has better performance than the other 5 comparison algorithms on 28 out of 40 test instances. In particular, when the number of objectives increases, the proposed *PEA* shows better performance. For instance, algorithm *PEA* has lower *IGD* values on 6 out of the 10 5-objective test instances, which generates better performance on 8 out of the 10 10-objective test instances. This can be explained that the five comparison algorithms follow the traditional framework of MOEAs, and try to balance the convergence and diversity during each iteration.

TABLE 1. COMPARISON OF IGD AMONG THE 6 ALGORITHMS ON BENCHMARKS LSMOP1-LSMOP3 AND MAF1-MAF3

Problems	m	<i>LMEA</i>	<i>MOEA/DVA</i>	<i>NSGA-III</i>	<i>MaOEA-R&D</i>	<i>BiGE</i>	<i>PEA</i>
LSMOP1	3	0.072 (8.9e-03)–	0.047 (6.8e-04)+	0.118 (8.9e-03)–	0.716 (1.9e-01)–	0.339 (4.4e-02)–	0.048 (1.1e-03)
	5	0.200 (2.3e-02)–	0.198 (3.0e-02)–	0.254 (7.1e-02)–	0.764 (9.1e-02)–	0.439 (2.8e-02)–	0.151 (4.9e-03)
	8	0.355 (2.6e-02)–	0.344 (5.0e-06)–	0.686 (1.1e-01)–	8.413 (3.7e+00)–	0.567 (5.7e-02)–	0.227 (3.8e-03)
	10	0.434 (2.6e-02)–	0.399 (1.1e-03)–	1.244 (2.2e-01)–	7.079 (3.2e+00)–	0.813 (1.0e-01)–	0.238 (3.7e-03)
LSMOP2	3	0.079 (1.5e-02)–	0.073 (1.8e-03)–	0.060 (2.2e-03)≈	0.099 (2.9e-03)–	0.232 (6.8e-02)–	0.059 (1.0e-02)
	5	0.201 (1.5e-01)–	0.178 (8.4e-04)–	0.153 (1.1e-03)+	0.254 (4.3e-02)–	0.432 (6.2e-02)–	0.154 (8.8e-03)
	8	0.658 (7.4e-02)–	0.397 (5.0e-05)–	0.257 (1.1e-03)–	0.410 (5.9e-02)–	0.561 (3.2e-02)–	0.224 (4.7e-03)
	10	0.731 (3.1e-02)–	0.357 (8.6e-03)–	0.307 (1.8e-03)–	0.394 (4.1e-02)–	0.488 (2.3e-02)–	0.230 (4.6e-03)
LSMOP3	3	0.429 (5.4e-02)+	0.447 (2.3e-02)+	0.587 (7.1e-02)–	0.810 (1.2e-01)–	0.669 (2.6e-01)–	0.454 (8.2e-02)
	5	0.519 (1.0e-01)+	0.766 (4.0e-02)–	1.288 (4.1e-01)–	0.948 (2.1e-02)–	0.877 (3.5e-01)–	0.565 (1.2e-01)
	8	1.978 (4.1e-01)–	1.548 (5.9e-02)–	3.157 (2.5e+00)–	0.881 (1.1e-01)–	4.699 (2.1e+00)–	0.687 (1.2e-01)
	10	5.367 (1.3e+00)–	3.627 (2.0e+00)–	2.180 (5.5e-01)–	2.166 (2.7e+00)–	3.729 (8.6e-01)–	0.682 (2.1e-01)
MaF1	3	0.053 (4.2e-03)–	0.076 (4.6e-04)–	0.067 (1.2e-03)–	0.399 (2.0e-02)–	0.067 (7.3e-03)–	0.048 (7.0e-04)
	5	0.158 (6.9e-03)–	0.185 (8.2e-05)–	0.394 (2.9e-02)–	0.463 (8.5e-02)–	0.180 (3.7e-02)–	0.146 (1.5e-03)
	8	0.330 (1.6e-02)–	0.365 (1.1e-02)–	0.398 (2.2e-02)–	0.544 (3.9e-02)–	0.245 (8.9e-03)–	0.224 (1.4e-03)
	10	0.347 (1.2e-02)–	0.680 (2.8e-02)–	0.367 (9.5e-03)–	0.579 (5.1e-02)–	0.270 (1.1e-02)–	0.238 (1.8e-03)
MaF2	3	0.034 (1.1e-03)≈	0.047 (4.1e-04)–	0.042 (6.4e-04)–	0.321 (5.5e-02)–	0.090 (1.9e-02)–	0.034 (6.3e-04)
	5	0.100 (1.5e-03)+	0.099 (4.6e-06)+	0.177 (5.7e-03)–	0.537 (6.8e-03)–	0.231 (1.6e-02)–	0.124 (2.7e-03)
	8	0.267 (1.4e-02)–	0.345 (2.8e-02)–	0.441 (4.7e-02)–	0.811 (4.3e-02)–	0.219 (1.0e-02)–	0.169 (5.0e-03)
	10	0.293 (1.3e-02)–	0.390 (5.4e-02)–	0.442 (3.6e-02)–	0.858 (9.4e-03)–	0.212 (6.9e-03)–	0.180 (5.2e-03)
MaF3	3	0.066 (3.1e-03)–	54.88 (1.5e+01)–	0.052 (3.1e-04)+	0.286 (4.9e-02)–	0.189 (4.5e-02)–	0.054 (3.3e-03)
	5	0.112 (2.5e-03)–	9.086 (4.8e+00)–	0.111 (3.1e-03)–	0.413 (7.1e-02)–	0.201 (4.1e-02)–	0.099 (2.9e-03)
	8	0.127 (2.8e-03)+	82.94 (2.7e+01)–	5.9e+10 (3e+10)–	0.402 (8.5e-02)–	2.2e+09 (6e+09)–	0.161 (5.5e-02)
	10	0.104 (2.0e-03)–	583.8 (1.7e+02)–	8.7e+11 (5e+11)–	0.475 (1.2e-01)–	1.5e+10 (2e+10)–	0.097 (5.7e-02)
MaF4	3	0.366 (1.9e-02)–	18.65 (7.1e+00)–	0.400 (2.2e-02)–	2.889 (1.1e-01)–	1.594 (2.1e-01)–	0.339 (1.8e-02)
	5	2.614 (7.7e-02)–	40.18 (9.6e+00)–	889.0 (2.5e+02)–	12.64 (1.1e+00)–	6.061 (6.1e-01)–	2.547 (2.1e-01)
	8	18.19 (1.6e+00)–	813.9 (1.5e+02)–	39.26 (1.4e+00)–	126.4 (8.6e+00)–	53.61 (9.9e+00)–	17.85 (3.2e+00)
	10	50.98 (2.1e+00)+	7954 (9.5e+02)–	152.8 (1.1e+01)–	517.9 (4.5e+01)–	191.6 (2.5e+01)–	59.21 (6.6e+00)
MaF5	3	0.507 (7.5e-01)–	0.714 (4.9e-01)–	0.870 (1.2e+00)–	0.402 (3.8e-01)–	0.859 (1.4e+00)–	0.283 (3.8e-03)
	5	4.893 (4.5e+00)–	4.069 (3.0e-01)–	2.802 (9.8e-01)–	3.716 (1.6e+00)–	3.147 (1.3e-01)–	2.431 (5.6e-02)
	8	51.36 (3.4e+01)–	54.86 (1.5e+01)–	24.62 (1.3e+00)–	18.10 (5.0e+00)–	21.56 (1.9e+00)–	16.75 (7.5e-01)
	10	172.3 (1.3e+02)–	150.5 (1.7e+01)–	87.73 (1.1e+00)–	60.26 (2.4e+01)+	97.12 (1.5e+01)–	62.88 (3.2e+00)
MaF6	3	0.011 (1.3e-03)–	0.347 (2.0e-02)–	0.021 (2.6e-03)–	0.742 (1.0e-05)–	0.024 (5.0e-03)–	0.006 (3.8e-05)
	5	0.012 (1.9e-03)–	4.130 (5.7e+00)–	0.111 (2.4e-02)–	0.742 (5.8e-05)–	0.049 (1.4e-02)–	0.006 (1.4e-05)
	8	0.019 (1.7e-03)–	12.02 (2.7e+01)–	0.120 (2.3e-02)–	0.743 (1.2e-04)–	1.046 (4.3e-01)–	0.005 (4.0e-05)
	10	0.402 (3.4e-01)–	0.166 (3.0e-02)–	0.089 (2.0e-02)–	0.745 (5.9e-04)–	5.586 (4.5e-01)–	0.003 (3.3e-05)
MaF7	3	0.101 (1.2e-02)–	0.143 (6.4e-03)–	0.087 (2.9e-03)–	1.156 (3.3e-01)–	0.108 (9.8e-03)–	0.068 (1.5e-03)
	5	0.368 (1.1e-02)≈	0.387 (5.0e-04)–	0.391 (2.1e-02)–	2.648 (4.1e-01)–	0.431 (5.2e-02)–	0.368 (8.2e-03)
	8	0.745 (1.4e-02)–	0.814 (1.6e-02)–	0.793 (2.2e-02)–	3.297 (1.0e+00)–	2.278 (3.2e-01)–	0.726 (2.5e-02)
	10	0.956 (1.4e-02)–	1.269 (4.4e-02)–	1.030 (8.0e-02)–	3.680 (8.7e-01)–	3.258 (3.9e-01)–	0.899 (2.0e-02)

However, when increasing the number of objectives, the ratio of non-dominated solutions in a population rises rapidly, and these solutions become incomparable in terms of the Pareto dominance relation, which makes these traditional algorithm lose the ability to distinguish the solutions in a population. Besides, the objective space expands exponentially with the increase of the number of objectives, and the limited number of solutions becomes more sparse in the objective space, which poses higher challenges for these algorithms to balance the convergence and diversity.

Being different from these algorithms, the proposed *PEA* separates the aspects of convergence and diversity, and each subpopulation merely emphasizes at the convergence while the diversity is pursued among the converged solutions, such that avoiding the negative effects caused by the conflicts between convergence and diversity.

To visually illustrate the distribution of solutions obtained by different algorithms, the solutions in final populations of the six algorithms on two test instances (i.e., 3-objective LSMOP1 and 10-objective MaF1) are shown in

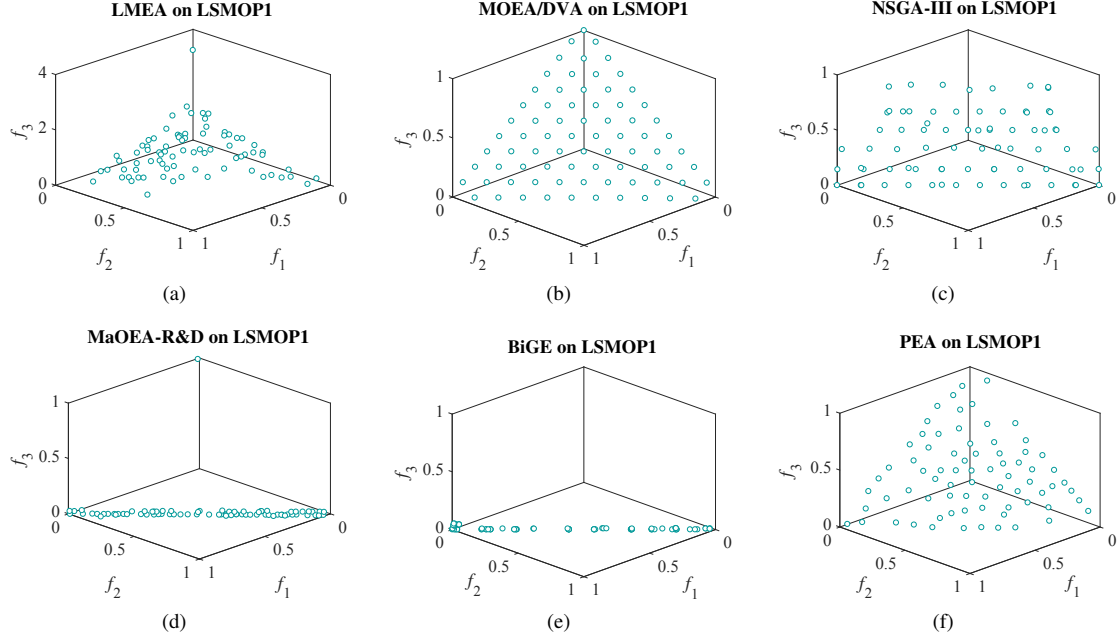


Figure 4. The Solutions with the Lowest IGD Values Obtained by 6 Algorithms Among 30 Runs in the Objective Space on 3-objective LSMOP1.

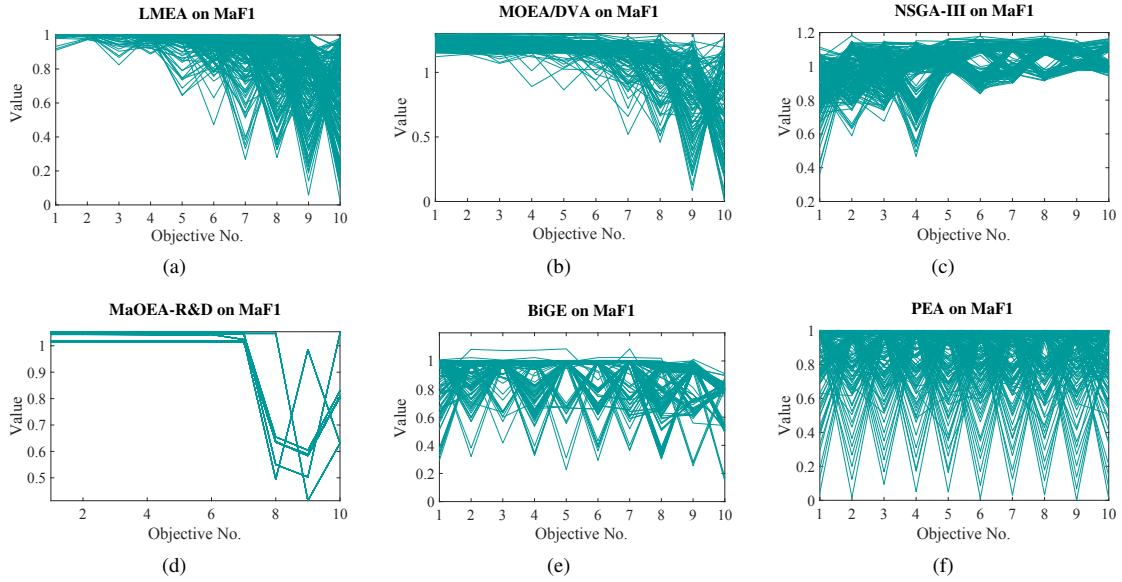


Figure 5. The Solutions with the Lowest IGD Values Among 30 Runs of the 6 Algorithms on 10-objective MaF1, Shown by Parallel Coordinates.

Figs. 4 and 5.

For the *PF* of benchmark function LSMOP1 [4], it ranges from 0 to 1 in each objective dimension. From Fig. 4, we can see that the algorithm *LMEA* does not converge on the third objective. The algorithm *MOEA/DVA* has good performance with respect to both the convergence and diversity, which can explain that it has the lowest *IGD* value in the test instance 3-objective LSMOP1. The other three comparison algorithms, *NSGA-III*, *MaOEA-R&D* and *BiGE*, all have good convergence, but poor diversity in the third objective. When we look at Fig. 4 (f), we can see that the distribution of the output solutions of the proposed *PEA*

is similar to that of algorithm *MOEA/DVA*. This is the reason that the proposed *PEA* has similar *IGD* value to *MOEA/DVA*, which can be observed in the first row of Table 1.

To visually compare the performance the six algorithms in terms of convergence and diversity on test instances with high-dimensional objectives, the output solutions of the six algorithms on 10-objective MaF1 are shown in Fig. 5. The *PF* of benchmark function MaF1 [23] ranges from 0 to 1 in each objective dimension. On this test instance, *LMEA* shows good performance in terms of convergence, but loses the diversity on the first eight objectives, which is shown in Fig. 5 (a). For algorithms *MOEA/DVA*, *NSGA-III* and

TABLE 2. EXECUTION TIME (UNIT: S) OF THE 6 ALGORITHMS ON 3-OBJECTIVE LSMOP1 WITH 3.07×10^6 FUNCTION EVALUATIONS

	<i>LMEA</i>	<i>MOEA/DVA</i>	<i>NSGA-III</i>	<i>MaOEA-R&D</i>	<i>BiGE</i>	<i>PEA</i>
Seq.	5.931e+02	1.288e+03	3.420e+02	2.968e+02	3.094e+02	1.303e+02
2 cores	3.425e+02 (1.732)	7.438e+02 (1.732)	2.502e+02 (1.367)	1.845e+02 (1.609)	2.093e+02 (1.478)	7.296e+01 (1.800)
4 cores	1.876e+02 (3.162)	4.073e+02 (3.162)	1.871e+02 (1.827)	1.133e+02 (2.616)	1.438e+02 (2.151)	3.648e+01 (3.531)
6 cores	1.359e+02 (4.364)	2.952e+02 (4.364)	1.662e+02 (2.058)	8.978e+01 (3.306)	1.219e+02 (2.537)	2.532e+01 (5.195)
8 cores	1.101e+02 (5.387)	2.391e+02 (5.387)	1.556e+02 (2.197)	7.795e+01 (3.808)	1.112e+02 (2.786)	1.924e+01 (6.797)

MaOEA-R&D, they illustrate poor performance in both convergence and diversity. Note that algorithm *MaOEA-R&D* converges to several solutions. Compared with the above algorithms, *BiGE* has better performance, but its convergence and diversity still have a lot of room for improvement. As shown in Fig. 5 (f), the proposed *PEA* generates better performance with respect to both the convergence and diversity, which can be explained why the *IGD* value significantly outperforms other 5 comparison algorithms on this test instance. This result demonstrates that *PEA* is scalable in both the number of decision variables and objectives.

3.3. Computational speedup

This subsection compares the performance of the proposed *PEA* with the 5 comparison algorithms in terms of their speedups. Since the 5 comparison algorithms are sequential in their original version, we employ two widely-used parallel models to parallelize them. This first model is dimension-parallel, which divides the problem dimensions into multiple partitions and distributes them on different workers; while the second model is population-parallel, which divides the populations into many subpopulations and distributes them on different workers [15]. For the algorithms *LMEA* and *MOEA/DVA*, they divide the dimensions of decision variables into a series of subcomponents and optimize these subcomponents respectively. So, the dimension-parallel model is good for parallelizing them, and we employ this model on these two algorithms. For the other three algorithms, *NSGA-III*, *MaOEA-R&D* and *BiGE*, we employ the population-parallel model to parallelize them, and the number of subpopulations is equal to the number of the available cores.

In this paper, the speedup is defined as the ratio of the execution time using sequential model and the execution time using parallel model. Table 2 summarizes the computational time and speedups (in parentheses) of the 6 algorithms on 3-objective LSMOP1 with respect to the sequential version and the parallel versions on 2, 4, 6 and 8 cores.

As shown in Table 2, the execution time of the proposed *PEA* is much less than other 5 comparison algorithms in sequential model. For instance, the execution time of *PEA* is about 10 percent of that of *MOEA/DVA* with respect to the sequential model, which is shown in the first row of Table 2. The reason is that the environmental selection is time consuming, but the proposed *PEA* separates the con-

vergence and diversity, and environmental selection operator is only employed on the converged solutions, while it will be operated to balance the convergence and diversity during iterations in the other five comparison algorithms. This interpretation can also explain that the execution time of *PEA* is much less than other 5 comparison algorithms in parallel model with different cores. In addition, we can also observe that the speedup of the proposed *PEA* is much higher than that of other algorithms, especially for the cases with more cores. This can be attributed to the fact that each subpopulation in *PEA* is highly independent and parallel, and only returns a solution with the best fitness after converging. By doing that, the time overhead of data exchanging among different worker can be avoided. These results demonstrate the advantage of *PEA* in reducing the time of solving large-scale multi-objective optimization problems.

4. Related Studies

Driven by the practical demands encountered in various applications, a variety of MOEAs have been put forward and improved during the past two decades. Most of these MOEAs focus on environmental selection strategies to balance convergence and diversity of the population. Among them, there are roughly three general categories based on the environmental selection strategies [10], [13]: 1) Pareto-based; 2) indicator-based; 3) decomposition-based. For Pareto-based MOEAs, they classify all the candidate solutions in the population into many levels on the basis of their Pareto dominance relations, and solutions in former levels are preferentially selected. Then, the solutions in the last accepted level will be sorted by a second feature, e.g., crowding distance [20], perpendicular distance to the reference vectors [9], ϵ [25], grid-based criteria [26] and the like. The indicator-based MOEAs use one or two indicators to sort all the candidate solutions, and the representative works of this category are HyPE [27], BiGE [5] and others. Regarding the decomposition-based MOEAs, they decompose an MOP into a series of sub-problems and solve them collaboratively [28], [29]. The representative of this category is the algorithm MOEA/D proposed by Zhang *et al.* [28]. Then, many strategies are proposed to improve the performance of the original version of MOEA/D. For instance, Asafuddoula *et al.* employed adaptive weight vectors in MOEA/D to handle MOPs with irregular PFs [30]. Another branch of the decomposition based MOEAs is to

decompose the objective space into a series of subspaces by a set of reference vectors and maintain a subpopulation in each subspace collaboratively. Representative works along with this branch are MOEA/D-M2M [31] and RVEA [32]. However, these MOEAs are serial and designed for MOPs with low dimensional variables (no more than 50).

So far, there also exist several MOEAs for solving large-scale MOPs. For instance, Antonio *et al.* designed a cooperative coevolution framework to solve large-scale MOPs [33]. The framework first randomly divides the large number of decision variables into several small subcomponents, and then coevolve these subcomponents cooperatively [33]. Zille *et al.* proposed a weighted optimization framework, which extends traditional MOEAs to include a problem transformation process that divides the decision variables into many groups, and all the variables in a group correspond to the same weight, then uses traditional MOEAs to optimize the weights instead of the decision variables [34]. Ma *et al.* suggested to divide the decision variables into diversity-related and convergence-related variables, then further divide convergence-related into many subcomponents based on the interactions among these variables. After that, the variables in each subcomponent are evolved until converging to the *PF*, and then the diversity-related variables are evolved to improve diversity [7]. To further accurately distinguish diversity-related and convergence-related variables, Zhang *et al.* employed *k*-means method to cluster the variables. These large-scale MOEAs also follow the framework of traditional MOEAs to balance convergence and diversity during each iteration, and have limits in parallel operation.

To reduce the execution time of MOEAs, two classical parallel frameworks in solving single-objective problems have been extended to parallelize MOEAs [15]. The first one is the population-parallel framework, which divides the whole population into many sub-populations, and distributed each sub-populations to different workers. For instance, the population-parallel model was used to speed up the the non-dominated sorting GA (NSGA-II) [35]. Durillo *et al.* employed this parallel model to distribute the MOEA/D to different workers for operating in parallel [36]. The second one is the dimension-parallel, which divides the variables into many subcomponents and evolves different subcomponents on different workers. For example, Cao *et al.* designed a distributed cooperative coevolutionary algorithm to evolve each subcomponent of variables in different workers [37]. However, in both existing frameworks, during each iteration, each sub-population (subcomponents) needs to exchange the newest solutions with other sub-populations, or the newest solutions in each sub-population (subcomponent) will be collected for environmental selection, which results in frequent data transmission and evolutionary processes on different dependent workers.

Unlike the traditional framework that balances convergence and diversity during each iteration, this paper develops a novel parallel framework that separates the convergence and diversity, and the convergence is performed by a series of subpopulations while the diversity is emphasized on the converged solutions from each subpopulation. By doing

that, the environmental selection operator has no impact on the evolutionary process of each subpopulation, and the dependencies during the population evolution are removed.

5. Conclusions and Future Work

This work designed a parallel framework that removes the dependencies among the sub-processes of the whole evolutionary process, enhancing the aspect of parallelism of the approach. On the basis of this framework, a parallel evolutionary algorithm was proposed to separate the convergence and diversity. The convergence was achieved by a series of subpopulations without communicating with each other, and an environmental selection strategy was proposed to select the converged solutions from each subpopulation. Then, extensive experiments on 10 benchmark functions with 3, 5, 8 and 10 objectives were conducted to compare the performance of the proposed algorithm with five state-of-the-art multi-objective evolutionary algorithms. In terms of the IGD indicator, the proposed PEA significantly outperforms the five comparison algorithms on 28 out of 40 test instances. Additionally, the performance of the proposed parallel framework in terms of speedup was also compared with the two widely-used parallel models. The experimental results demonstrated that the proposed PEA prevails over existing algorithms with respect to the speedup.

The convergence velocities of different decision variables in MOPs vary significantly. Having this in mind, the problem of how to adaptively allocate computational resource for different decision variables is an interesting direction of future research. In addition, cloud has become a promising paradigm that enables elastic provisioning of computational resources, then the problem of how to adopt the parallelism of the parallel framework to the available resources is another valuable direction.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grant 61572511, in part by the Scientific Research Project of National University of Defense Technology under Grant ZK16-03-57 and Grant ZK16-03-09, in part by the China Postdoctoral Science Foundation under Grant 2016M602960 and Grant 2017T100796.

References

- [1] C. Wang, J. Li, F. Ye, and Y. Yang, "Improve charging capability for wireless rechargeable sensor networks using resonant repeaters," in *Proceedings of the 35th IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2015, pp. 133–142.
- [2] H. Chen, X. Zhu, D. Qiu, L. Liu, and Z. Du, "Scheduling for workflows with security-sensitive intermediate data by selective tasks duplication in clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 9, pp. 2674–2688, 2017.
- [3] Y. Qi, L. Bao, X. Ma, Q. Miao, and X. Li, "Self-adaptive multi-objective evolutionary algorithm based on decomposition for large-scale problems: A case study on reservoir flood control operation," *Information Sciences*, vol. 367, pp. 529–549, 2016.

- [4] R. Cheng, Y. Jin, M. Olhofer, and B. Sendhoff, "Test problems for large-scale multiobjective and many-objective optimization," *IEEE Transactions on Cybernetics*, vol. 47, no. 12, pp. 4108–4121, 2017.
- [5] M. Li, S. Yang, and X. Liu, "Bi-goal evolution for many-objective optimization problems," *Artificial Intelligence*, vol. 228, pp. 45–65, 2015.
- [6] X. Zhang, Y. Tian, R. Cheng, and Y. Jin, "A decision variable clustering-based evolutionary algorithm for large-scale many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 1, pp. 97–112, 2018.
- [7] X. Ma, F. Liu, Y. Qi, X. Wang, L. Li, L. Jiao, M. Yin, and M. Gong, "A multiobjective evolutionary algorithm based on decision variable analyses for multiobjective optimization problems with large-scale variables," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 2, pp. 275–298, 2016.
- [8] S. Bandaru, A. H.C. Ng, and K. Deb, "Data mining methods for knowledge discovery in multi-objective optimization: Part A - Survey," *Expert Systems With Applications*, vol. 70, pp. 139–159, 2017.
- [9] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints," *IEEE Transactions Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.
- [10] B. Li, J. Li, K. Tang, and X. Yao, "Many-objective evolutionary algorithms: A survey," *ACM Computing Surveys (CSUR)*, vol. 48, no. 1, p. 13, 2015.
- [11] P. Guo, X. Liu, T. Tang, S. Tang, and J. Cao, "Practical concurrent wireless charging scheduling for sensor networks," in *Proceedings of the 36th IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2016, pp. 741–742.
- [12] S. Tang, Y. Zhou, K. Han, Z. Zhang, J. Yuan, and W. Wu, "Networked stochastic multi-armed bandits with combinatorial strategies," in *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2017, pp. 786–793.
- [13] A. Zhou, B.-Y. Qu, H. Li, S.-Z. Zhao, P. N. Suganthan, and Q. Zhang, "Multiobjective evolutionary algorithms: A survey of the state of the art," *Swarm and Evolutionary Computation*, vol. 1, no. 1, pp. 32–49, 2011.
- [14] A. Atashpendar, B. Dorronsoro, G. Danoy, and P. Bouvry, "A scalable parallel cooperative coevolutionary pso algorithm for multi-objective optimization," *Journal of Parallel and Distributed Computing*, vol. 112, pp. 111–125, 2018.
- [15] Y.-J. Gong, W.-N. Chen, Z.-H. Zhan, J. Zhang, Y. Li, Q. Zhang, and J.-J. Li, "Distributed evolutionary algorithms and their models: A survey of the state-of-the-art," *Applied Soft Computing*, vol. 34, pp. 286–300, 2015.
- [16] N. Wang, D. Yu, H. Jin, C. Qian, X. Xie, and Q.-S. Hua, "Parallel algorithm for core maintenance in dynamic graphs," in *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2017, pp. 2366–2371.
- [17] M. Raynal, "A look at basics of distributed computing," in *Proceedings of the 36th IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2016, pp. 1–11.
- [18] S. Das and P. N. Suganthan, "Differential evolution: A survey of the state-of-the-art," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, 2011.
- [19] G. Wu, X. Shen, H. Li, H. Chen, A. Lin, and P. Suganthan, "Ensemble of differential evolution variants," *Information Sciences*, vol. 367, pp. 172–186, 2018.
- [20] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [21] Z. He and G. G. Yen, "Many-objective evolutionary algorithm: Objective space reduction and diversity improvement," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 1, pp. 145–160, 2016.
- [22] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "PlatEMO: A matlab platform for evolutionary multi-objective optimization [educational forum]," *IEEE Computational Intelligence Magazine*, vol. 12, no. 4, pp. 73–87, 2017.
- [23] R. Cheng, M. Li, Y. Tian, X. Zhang, S. Yang, Y. Jin, and X. Yao, "A benchmark test suite for evolutionary many-objective optimization," *Complex & Intelligent Systems*, vol. 3, no. 1, pp. 67–81, 2017.
- [24] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 4, pp. 257–271, 1999.
- [25] D. Hadka and P. Reed, "Borg: An auto-adaptive many-objective evolutionary computing framework," *Evolutionary Computation*, vol. 21, no. 2, pp. 231–259, 2013.
- [26] S. Yang, M. Li, X. Liu, and J. Zheng, "A grid-based evolutionary algorithm for many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 17, no. 5, pp. 721–736, 2013.
- [27] J. Bader and E. Zitzler, "Hype: An algorithm for fast hypervolume-based many-objective optimization," *Evolutionary Computation*, vol. 19, no. 1, pp. 45–76, 2011.
- [28] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.
- [29] A. Trivedi, D. Srinivasan, K. Sanyal, and A. Ghosh, "A survey of multiobjective evolutionary algorithms based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 21, no. 3, pp. 440–462, 2017.
- [30] M. Asafuddoula, H. K. Singh, and T. Ray, "An enhanced decomposition-based evolutionary algorithm with adaptive reference vectors," *IEEE Transactions on Cybernetics*, 2017, DOI:10.1109/TCYB.2017.2737519.
- [31] H.-L. Liu, F. Gu, and Q. Zhang, "Decomposition of a multiobjective optimization problem into a number of simple multiobjective subproblems," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 3, pp. 450–455, 2014.
- [32] R. Cheng, Y. Jin, M. Olhofer, and B. Sendhoff, "A reference vector guided evolutionary algorithm for many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 5, pp. 773–791, 2016.
- [33] L. M. Antonio and C. A. C. Coello, "Use of cooperative coevolution for solving large scale multiobjective optimization problems," in *Proceedings of the 2013 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2013, pp. 2758–2765.
- [34] H. Zille, H. Ishibuchi, S. Mostaghim, and Y. Nojima, "A framework for large-scale multi-objective optimization based on problem transformation," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 2, pp. 260–275, 2018.
- [35] G. Ewald, W. Kurek, and M. A. Brdys, "Grid implementation of a parallel multiobjective genetic algorithm for optimized allocation of chlorination stations in drinking water distribution systems: Chojnice case study," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 4, pp. 497–509, 2008.
- [36] J. J. Durillo, Q. Zhang, A. J. Nebro, and E. Alba, "Distribution of computational effort in parallel MOEA/D," in *LION*. Springer, 2011, pp. 488–502.
- [37] B. Cao, J. Zhao, Z. Lv, and X. Liu, "A distributed parallel cooperative coevolutionary multi-objective evolutionary algorithm for large-scale optimization," *IEEE Transactions on Industrial Informatics*, vol. 13, no. 4, pp. 2030–2038, 2017.